Operations on Doubly Linked List

In a Doubly Linked List (DLL) each node contains a data field, a reference to the next node in the sequence (next), and a reference to the preceding node in the sequence (prev). The operations we are going to discuss are:

1. **Insert**: Adds a new node to the list. It can be at the beginning, end, or any specific position.
2. **Insert After**: Adds a new node after a specified node.
3. **Delete**: Removes a node with the specified data from the list.
4. **Search**: Searches for a node with the specified data in the list and returns true if found, otherwise false.

Java

```java
// Define a DLNode class to represent elements in the doubly linked list.
// class DLNode {
//     int val;
//     DLNode prev;
//     DLNode next;

//     // Constructor to initialize a DLNode with a given value.
//     DLNode(int d) {
//         val = d;
//         prev = next = null;  // Initialize previous and next pointers to null.
//     }
// }

// Define a Solution class to manage the doubly linked list operations.
class Solution {
    DLNode head;

    // Inserts a new DLNode at the front of the list.
    void insert(int val) {
        DLNode newNode = new DLNode(val);  // Create a new DLNode with the given value.
        newNode.next = head;
        if (head != null)
            head.prev = newNode;
        head = newNode;  // Update the head to the new DLNode.
```

Run Code Result | Stdout

You must run your code first

Execute

← Previous

Operations On Singly Linked List

Next →

Problem 1: Reverse Linked List (Easy)

Mark as Completed